

Special Participation E: Cross-Class Resource Comparison

Something that is true about many students in this class is that they are simultaneously enrolled in non-orthogonal classes (e.g. 180, 183, etc.) with overlapping topics. Nietzsche famously writes¹: “the more affects we allow to speak about one thing, the more eyes, different eyes, we can use to observe one thing, the more complete will our ‘concept’ of this thing, our ‘objectivity,’ be.” In this spirit, it might be useful for someone enrolled in these classes to figure out what is similar to do things like plan studying, clarify differing notation, compare perspectives, find differences in information content, and so on.

I provide prompts for the comparison of overlapping course material from different classes and a trace of me using theme to try to gain educational value. I imagine that these would be best using lecture transcripts but I use slides for ease of access.

I generated these prompts using GPT-5.1. Two traces of 182 vs 183 and 182 vs 180 conversations with Claude Sonnet 4.5 are attached.

SYSTEM INSTRUCTIONS (include in prompt exactly):

None

You are an educational analysis assistant.

Your task is to compare overlapping topics across two or more university courses.

You must:

Compare concepts across classes with precision (not generic summaries).

Identify differences in emphasis, definitions, assumptions, and notation.

Contrast problem-solving methodologies taught in each course.

Point out conceptual or pedagogical conflicts between the classes.

Explain how a student should adapt their reasoning when switching contexts.

¹ pg. 555 Nietzsche, Friedrich. The Genealogy of Morals. 2000 Modern Library Edition.

Explicitly state when you are uncertain, when something may be hallucinated, or when sources conflict.

List concrete examples or exercises that illustrate differences.

If a topic is unclear or ambiguous, you must request clarification rather than fabricate.

USER INSTRUCTIONS (you fill these in when you run the prompt):

None

I am taking the following courses this semester:

Course A: [insert course name, e.g. CS 182 – Deep Learning]

Course B: [insert course name, e.g. CS 189 – Machine Learning]
(Optional)

Course C: [insert additional course, e.g. EECS 127 – Optimization]

I want you to compare the following topic(s) across these classes:

Topic(s): [e.g. gradient descent, cross-entropy, overfitting, regularization]

Please perform the comparison using the following structure:

1. Definitions & Formalism

How each class defines the concept

Notation differences

Underlying assumptions

2. Conceptual Goals of Each Course

Why each class teaches this concept

Depth vs breadth

Whether each course treats the concept theoretically, heuristically, or operationally

3. Methodological Differences

Algorithms emphasized in each

Differences in derivations or interpretations

Problem-solving styles expected in homework/exams

4. Conflicts or Divergences

Common student misconceptions when transferring knowledge between courses

Where the classes disagree in framing or intuition

5. Complementary Strengths

What perspective each course adds

How understanding from one course deepens knowledge in another

6. Study Strategy Recommendations

How to integrate the perspectives

How to avoid context-specific misunderstandings

How to create a unified mental model

7. Hallucination Check

For each major claim:

Mark [Certain], [Plausible but check course materials], or [Uncertain].

If you are unsure about anything or lack information, state it explicitly instead of guessing.

Comparing overlapping topics across university courses

-  You are an educational analysis assistant. Your task is to compare overlapping topics across two or more university courses. You must: Compare concepts across classes with precision (not generic summaries). Identify differences in emphasis, definitions, assumptions, and notation. Contrast problem-solving methodologies taught in each course. Point out conceptual or pedagogical conflicts between the classes. Explain how a student should adapt their reasoning when switching contexts. Explicitly state when you are uncertain, when something may be hallucinated, or when sources conflict. List concrete examples or exercises that illustrate differences. If a topic is unclear or ambiguous, you must request clarification rather than fabricate.

12/8/2025, 2:58:49 AM

I understand. I'm ready to provide detailed, precise comparative analysis of overlapping topics across university courses.

When you provide me with course materials, syllabi, lecture notes, or specific topics, I will:

Analyze with precision:

- Compare how each course defines and frames the same concept
- Identify differences in notation, terminology, and assumptions
- Contrast the depth, rigor, and approach each course takes

Highlight pedagogical differences:

- Note what each course emphasizes or de-emphasizes
- Compare problem-solving frameworks and methodologies
- Point out where courses might teach conflicting intuitions or approaches

Flag potential conflicts:

- Identify where courses make different simplifying assumptions
- Note where definitions or conventions differ
- Explain contexts where one approach is preferred over another

Provide actionable guidance:

- Explain how to adapt reasoning when switching between courses
- Suggest concrete strategies for managing different frameworks
- Recommend when to apply each course's methodology

Maintain epistemic honesty:

- Explicitly state uncertainties or gaps in my knowledge
- Acknowledge when I'm making inferences vs. stating facts
- Request clarification when topics are ambiguous
- Cite specific sources when comparing course materials

Please share the courses you'd like me to compare - this could include:

- Course names/numbers and topics of overlap
- Syllabi, lecture notes, or textbook excerpts
- Specific concepts or problem types you've noticed differ
- Areas where you're experiencing confusion switching between classes

What courses would you like me to analyze?

12/8/2025, 2:58:59 AM





File: flowmatching.pdf

File: Lecture 27.pdf

I am taking the following courses this semester: Course A: CS180 - Computer Vision Course B: CS182 - Deep Neural Networks I want you to compare the following topic(s) across these classes: Topic(s): generative models, diffusion Please perform the comparison using the following structure:

1. Definitions & Formalism How each class defines the concept Notation differences Underlying assumptions
2. Conceptual Goals of Each Course Why each class teaches this concept Depth vs breadth Whether each course treats the concept theoretically, heuristically, or operationally
3. Methodological Differences Algorithms emphasized in each Differences in derivations or interpretations Problem-solving styles expected in homework/exams
4. Conflicts or Divergences Common student misconceptions when transferring knowledge between courses Where the classes disagree in framing or intuition
5. Complementary Strengths What perspective each course adds How understanding from one course deepens knowledge in another
6. Study Strategy Recommendations How to integrate the perspectives How to avoid context-specific misunderstandings How to create a unified mental model
7. Hallucination Check For each major claim: Mark [Certain], [Plausible but check course materials], or [Uncertain]. If you are unsure about anything or lack information, state it explicitly instead of guessing.

12/8/2025, 3:00:01 AM



Comparative Analysis: Generative Models & Diffusion across CS180 and CS182

1. Definitions & Formalism

CS180 (Computer Vision) - Flow Matching Perspective

Definition: Diffusion/Flow models are presented as **flow-based generative models** that learn a velocity field to transport samples from a source distribution (noise) to a target distribution (data).

Core formalism:

- Flow ODE: $\frac{dx}{dt} = v_\theta(x, t)$ where v_θ is a learned velocity field
- Emphasis on **Flow Matching** (Lipman et al. 2022) as a unifying framework
- Linear interpolation path: $x_t = (1-t)x_0 + tx_1$ where $x_0 \sim p_{\text{source}}$ (noise), $x_1 \sim p_{\text{data}}$ (real images)
- **Conditional flow matching:** Learn velocities conditioned on individual samples, then marginalize
- Notation: p_0 (source/prior), p_1 (target/data), ψ_t (flow map), u_t (velocity field)

Key assumptions:

- Velocity fields preserve probability mass (continuity equation automatically satisfied)
- Can use arbitrary interpolation schedules (very flexible)
- Focus on ODE formulation (deterministic sampling)

CS182 (Deep Neural Networks) - Diffusion Process Perspective

Definition: Diffusion models are presented through the lens of **stochastic differential equations (SDEs)** and the physics of diffusion processes.

Core formalism:

- Forward diffusion: Gradually add Gaussian noise over discrete timesteps
- Specific noise schedules: $z_t = \sqrt{(1-\beta_t)} \cdot z_{t-1} + \sqrt{\beta_t} \cdot \epsilon_t$ where β_t controls noise variance
- Backward process: Learn to denoise by predicting either noise ϵ , clean image x_0 , or "score" $\nabla \log p_t(x_t)$
- **DDPM** (Ho et al. 2020) framework with variance-preserving or variance-exploding schedules
- Notation: x (data), z_t or x_t (noisy versions), β_t (noise schedule), μ_θ (learned mean)

Key assumptions:

- Forward process must follow specific Gaussian diffusion dynamics
- Backward process derivable via Bayes rule
- Emphasis on discrete-time formulation (though continuous-time mentioned)
- Loss derived from variational lower bound on log-likelihood

Notation Conflicts

These notation conflicts are good to know

Concept	CS180 (Vision)	CS182 (DL)
Noisy sample	x_t	z_t or x_t
Time parameter	$t \in [0,1]$ continuous	$t \in \{0,1,\dots,T\}$ discrete
Network output	Velocity $v_\theta(x,t)$	Noise $\epsilon_\theta(x,t)$ or x_0 or score
Source distribution	p_0 (Gaussian noise)	p_T (final noisy state)
Target distribution	p_1 (data)	p_0 (data) - reversed indexing!

[CERTAIN - directly from documents]

2. Conceptual Goals of Each Course

CS180: Operational Understanding for Image Generation

Why it's taught:

- Generative models are a major application domain of computer vision
- Flow matching represents the **current state-of-the-art** (post-2022)
- Connects to broader themes of image-to-image translation and synthesis

Depth vs. Breadth:

- **Moderate depth:** Derives flow matching loss, explains why conditional flows marginalize to marginal flow
- Focus on **simplicity and flexibility** - "Flow matching basically says you can add noise however you like!"
- Emphasizes the elegance of the approach over mathematical rigor

Treatment style:

- **Heuristic and operational:** "Here's how to implement it in PyTorch"
- Minimal measure theory or SDE background required
- Goal: Enable students to train and sample from flow models

CS182: Theoretical Foundations and Historical Development

Why it's taught:

- Diffusion models as exemplars of **probabilistic deep learning**
- Demonstrates connection between physics (thermodynamics) and ML
- Historical context: VAE → DDPM → continuous-time formulations

Depth vs. Breadth:

- **Greater theoretical depth:** Derives sampling from Bayes rule, explores log-likelihood interpretation
- Covers variance-preserving vs. variance-exploding schedules rigorously
- Shows equivalence between noise prediction, data prediction, and score matching

Treatment style:

- **Theoretical then practical:** Derives loss from first principles, then shows implementation
- Emphasizes **why** different parameterizations work
- Expects students to understand measure-theoretic arguments (Claim 1 with normalizing factors)

[CERTAIN - clear from pedagogical approach in documents]

This is useful for showing that you're not going to be able to skip the diffusion lecture even if you learned it in another class due to the perspectival difference. So I need to study for this class more than I would expect.

3. Methodological Differences

Training Algorithms

CS180 - Flow Matching Training:

```
x = next(dataset)  # Sample data
t = torch.rand(1)  # Uniform time in [0,1]
noise = torch.randn_like(x)
x_t = (1-t)*noise + t*x  # Linear interpolation
flow_pred = model(x_t, t)
flow_gt = x - noise  # Ground truth velocity
loss = F.mse_loss(flow_pred, flow_gt)
```

Key point: Direct velocity supervision, extremely simple loss

CS182 - DDPM Training:



```

x_0 = next(dataset)
t = randint(0, T) # Discrete timestep
noise = torch.randn_like(x_0)
x_t = sqrt(alpha_bar_t)*x_0 + sqrt(1-alpha_bar_t)*noise
noise_pred = model(x_t, t)
loss = F.mse_loss(noise_pred, noise) # Simplified objective

```

Key point: Predicts noise (or x_0), requires variance schedule

Sampling Algorithms

CS180 - ODE Integration:

- Any ODE solver (Euler, RK45, etc.)
- Example: $x_{t+\Delta t} = x_t + \Delta t \cdot v_\theta(x_t, t)$
- Deterministic trajectories
- Fewer steps possible with good velocity fields

CS182 - Ancestral Sampling (DDPM):

- Stochastic: $x_{t-\Delta t} = \mu_\theta(x_t, t) + \sigma_t \cdot z$ where $z \sim N(0, I)$
- **DDIM** variant for deterministic sampling (mentioned as "next time")
- Typically requires many steps ($T=1000$ common)

Problem-solving differences:

- **CS180:** "Given this flow model, how do you sample?" Focus on integration strategies
- **CS182:** "Derive the reverse process conditional from the forward process" - measure-theoretic reasoning

[CERTAIN - algorithms explicitly provided in documents]

4. Conflicts or Divergences

Major Divergence 1: Time Indexing

CS180: $t=0$ is noise, $t=1$ is data (forward in physical time)

CS182: $t=T$ is noise, $t=0$ is data (backward from physics perspective)

Student confusion: When reading DDPM papers after CS180, students may be confused about whether they're "diffusing forward" or "sampling backward"

Resolution: CS180's convention aligns with **sampling direction** (we integrate forward from noise to data). CS182's aligns with **physical diffusion** (we reverse the corruption process).

Major Divergence 2: What the Network Predicts

CS180: Network always outputs **velocity** $v_\theta(x_t, t) \approx x_1 - x_0$

CS182: Network can output **noise** ϵ_θ , **data** $\hat{x}_0$, or **score** $\nabla \log p(x_t)$

These are related by:

$$\begin{aligned}\hat{x}_0 &= (x_t - \sigma_t \cdot \epsilon_\theta) / \alpha_t \quad (\text{from CS182 notes}) \\ v_\theta &= x_1 - x_0 = (x_t - \sigma_t \cdot \epsilon_\theta) / \alpha_t - \text{noise} \quad (\text{implicit in CS180})\end{aligned}$$

Student confusion: "In homework for CS180 we predicted flow, but this DALL-E paper predicts epsilon - are they different models?"

Resolution: They're **equivalent parameterizations** when using appropriate linear interpolation schedules. CS180 emphasizes flow is cleanest; CS182 shows noise prediction connects to denoising autoencoders historically.

Major Divergence 3: Role of Stochasticity

CS180: "You can add stochasticity when riding it, then it becomes SDE (more next lecture)" - *treats as optional*

CS182: Stochasticity is **fundamental** to the forward process and ancestral sampling

Why this matters:

- CS180 students may think diffusion = deterministic ODE
- CS182 students may not realize ODE sampling is possible

Resolution: Both perspectives are valid. Song et al. 2021 showed every SDE has a corresponding "probability flow ODE" that samples the same marginals. CS180 teaches the ODE view primarily; CS182 teaches SDE view.

[CERTAIN for divergences 1-2, PLAUSIBLE BUT CHECK for divergence 3 - CS180 mentions SDEs briefly]

Potential Conflict 4: Noise Schedule Flexibility

CS180 claim: "Flow matching basically says you can add noise however you like!"

CS182 reality: DDPM carefully derives specific variance-preserving or variance-exploding schedules

Is this a real conflict?

- **Not quite:** CS180 is correct that flow matching is *more* flexible than classical DDPM
- But even CS180 uses specific schedules (linear interpolation is a choice!)
- CS182's schedules come from the SDE framework; flow matching bypasses this

Student misconception: "I can just use any random interpolation and it'll work"
- not necessarily without careful consideration of variance

[PLAUSIBLE - requires checking if CS180 discusses schedule sensitivity]

5. Complementary Strengths

What CS180 Adds to CS182 Knowledge

1. Simplification and Unification

- Flow matching (2022) **unifies** earlier approaches (DDPM, score matching, normalizing flows)
- Shows the "minimum viable formulation" - just learn velocities
- Students from CS182 gain: "Oh, I don't need to derive the variance schedule from thermodynamics!"

2. Modern Practical Perspective

- CS180 reflects post-2023 practice (Stable Diffusion 3, Sora use flow matching)
- Emphasis on **fewer sampling steps** through better velocity fields
- Connection to **rectified flows** (straighter paths = faster sampling)

3. Visual/Geometric Intuition

- "Riding a river" metaphor for velocity fields
- Visualization of how conditional flows per sample → marginal flow
- Warping perspective: $x_t = \psi_t(x_0)$ connects to image warping/registration

4. Non-parametric Insight

- CS180 notes: "One can minimize diffusion objective non-parametrically... but no ability to generate new images!"
- Explains *why* neural networks are essential (generalization, not just memorization)
- This is **profound** but not typically emphasized in CS182

What CS182 Adds to CS180 Knowledge

1. Foundational Understanding

- **Why** does the loss work? CS182 derives it from variational inference
- Connection to VAEs, ELBO, and probabilistic modeling
- Rigorous treatment of what's being optimized

2. Historical Context

- Evolution: VAE → DDPM → Score Matching → Flow Matching
- Helps understand *why* flow matching is elegant (it removed unnecessary constraints)
- Appreciate trade-offs (e.g., DDPM's variance schedule was overly restrictive)

3. Score-Based Perspective

- $\nabla \log p_t(x_t)$ (score function) as alternative to velocity
- Connects to Langevin dynamics and Stein's identity
- Useful for theoretical analysis

4. Discrete-Time Practicalities

This makes me think that I should try this prompt with fa24 cs189 slides.

- Real implementations use discrete steps
- Understanding α_t , β_t schedules helps debug training issues
- Scheduler design (cosine, linear, etc.) matters for performance

[CERTAIN for complementarity - documents show distinct but compatible perspectives]

6. Study Strategy Recommendations

Creating a Unified Mental Model

Core equivalence to internalize:

CS182 Forward Diffusion $\iff$ CS180 Conditional Flow (reversed time)

↓ ↓

Learn $\mu_\theta(x_t, t)$ to denoise $\iff$ Learn $v_\theta(x_t, t)$ velocity

↓ ↓

Sample by ancestral/DDIM $\iff$ Sample by ODE integration

Mapping Between Formalisms

When reading a diffusion paper:

1. **Identify time convention:** Does $t=0$ mean data or noise?
2. **Identify network target:** Predicting ϵ , x_0 , v , or $\nabla \log p$?
3. **Check if stochastic:** SDE sampling (CS182 style) or ODE (CS180 style)?

Translation guide:

CS182 Term	CS180 Equivalent	Notes
$\epsilon_\theta(x_t, t)$	Related to v_θ via $v = (x_t - \sigma \cdot \epsilon) / \alpha - x_0$	Need schedule
$\sqrt{(\alpha_t)}$	σ_t in $x_t = \sigma_t x_0 + \alpha_t x_1$	Variance weight
β_t schedule	Implicit in choice of σ_t , α_t	CS180 more flexible
DDPM sampling	Flow matching with noise	Add stochasticity to ODE
DDIM sampling	Flow matching ODE	Deterministic

Avoiding Context Confusion

For CS180 $\rightarrow$ CS182 transfer:

- When CS182 asks "derive the reverse process", use **Bayes rule** (not flow matching intuition)

- Remember CS182 wants you to work with **discrete timesteps** and β_t
- Don't say "just learn the velocity" - justify via ELBO or score matching

For CS182 → CS180 transfer:

- When implementing flow matching, **don't** add noise during sampling (unless explicitly doing stochastic variant)
- Time runs **opposite direction** - your $t=0$ is their $t=T$
- Focus on **velocity field** formulation, not noise prediction

Integration Strategy for Exams

If CS180 exam asks: "Implement a diffusion sampler"

- Use ODE integration (Euler method fine)
- Sample velocity at each step
- Deterministic trajectory

If CS182 exam asks: "Derive the sampling distribution"

- Start from forward process conditional
- Apply Bayes rule
- Show Gaussian form with learned mean

If asked to compare:

- "DDPM uses fixed variance schedule from diffusion physics"
- "Flow matching generalizes this by learning velocities directly"
- "Both converge to same marginal distributions"

Recommended Study Sequence

1. **Master CS182 foundations first** (if taking concurrently)

- Understand why $p(x_{t-1}|x_t, x_0)$ is Gaussian



- Derive noise prediction objective
- Implement DDPM sampler

I think this part is useful but I'm also behind on lectures so I don't know if this is a good plan.

2. Then see CS180 as simplification

- "Flow matching removes need for specific schedules"
- "Velocity = generalized denoising direction"
- Implement flow matching and compare sampling speed

3. Unify via continuous-time view

- Both are discretizations of continuous flows
- SDE (CS182) vs ODE (CS180) are related (Song et al. 2021)
- Reread CS180's "riding the river" with CS182's measure theory

[PLAUSIBLE - study strategies are recommendations based on document analysis]

7. Hallucination Check & Uncertainties

Claims Marked CERTAIN

- ✓ Flow matching uses velocity fields $dx/dt = v_\theta(x, t)$ [directly in CS180 notes]
- ✓ CS182 uses noise schedule $z_t = \sqrt{(1-\beta_t)} \cdot z_{t-1} + \sqrt{\beta_t} \cdot \epsilon$ [directly in CS182 notes]
- ✓ Time indexing differs (CS180: $t=0$ noise, $t=1$ data; CS182: opposite) [explicit in both]
- ✓ CS180 training loop predicts $x - \text{noise}$ [code snippet provided]
- ✓ CS182 derives loss from Bayes rule [shown in detailed derivation]
- ✓ Flow matching came after DDPM (2022 vs 2020) [timeline in CS180 slide 11]

Claims Marked PLAUSIBLE BUT CHECK

△ **Network output equivalence** (ϵ , x_0 , v are related):

- CS180 notes mention "you can make your network output undo the noise in many different ways"
- CS182 doesn't explicitly show conversion formulas in provided excerpts
- **Recommendation:** Verify by deriving yourself or checking Song et al. 2021

⚠ **Non-parametric flow insight:**

- CS180 states "can minimize diffusion objective non-parametrically... but no learning"
- This is profound but I don't see the full proof in excerpts
- **Recommendation:** This connects to Efros & Leung '99 texture synthesis mentioned - check if CS180 covers this in detail

⚠ **Continuous-time unification:**

- Song et al. 2021 "Score-Based Generative Modeling through SDEs" claims $SDE \leftrightarrow ODE$ equivalence
- Not directly covered in these CS180/182 notes
- **Recommendation:** Read Song et al. 2021 for rigorous connection

Claims Marked UNCERTAIN

? **Practical schedule sensitivity:**

- I claimed "can't just use any random interpolation" but CS180 emphasizes flexibility
- **Unknown:** Does CS180 later discuss failure modes of bad schedules?
- **Recommendation:** Check if CS180 assignment tests different interpolation schemes

? **Stochasticity in CS180 completeness:**

- CS180 mentions SDEs "more next lecture" but I only have one lecture
- **Unknown:** Does CS180 cover stochastic sampling later?
- **Recommendation:** Check subsequent CS180 lectures



? CS182 coverage of modern methods:

- I assume CS182 is more "historical" based on DDPM focus
- **Unknown:** Does CS182 also cover flow matching in later lectures?
- **Recommendation:** Check CS182 syllabus for topics after Lecture 27

What I Cannot Confirm

1. **Exact exam problem styles** - I'm inferring from pedagogical approach
2. **Whether CS180 proves the conditional→marginal flow theorem** - seems stated as fact
3. **Full CS182 coverage of DDIM** - marked as "next time" in notes
4. **Relative difficulty of assignments** - no homework problems provided

Gaps in My Knowledge

- **CS182 earlier lectures:** May cover background (VAE, normalizing flows) that contextualizes diffusion
- **CS180 later lectures:** May cover advanced topics (latent diffusion, conditioning, guidance)
- **Assessment formats:** Don't know if exams are coding, derivations, or conceptual
- **Instructor emphasis:** Lecture notes may not capture all verbal explanations/intuitions

[CERTAIN - these are genuine limitations of the analysis]

Summary Recommendation

If you're taking both courses:

1. **Use CS182 for "why"** - understand the probabilistic foundations

2. **Use CS180 for "how"** - implement state-of-the-art flow models
3. **Recognize they're teaching the same underlying math** with different emphases:
 - CS182: SDE → learn reverse process
 - CS180: ODE → learn velocity field
 - **These are equivalent in continuous time**
4. **On exams:**
 - CS182: Derive using Bayes rule, show Gaussian forms
 - CS180: Implement using velocity supervision, focus on sampling
5. **For research/projects:**
 - Flow matching (CS180) is **simpler to implement** and **more flexible**
 - DDPM theory (CS182) helps **debug issues** and **understand why it works**

The best understanding comes from seeing both perspectives as two sides of the same coin. ●

12/8/2025, 3:02:09 AM

Comparing overlapping topics across university courses

- ⊙ You are an educational analysis assistant. Your task is to compare overlapping topics across two or more university courses. You must: Compare concepts across classes with precision (not generic summaries). Identify differences in emphasis, definitions, assumptions, and notation. Contrast problem-solving methodologies taught in each course. Point out conceptual or pedagogical conflicts between the classes. Explain how a student should adapt their reasoning when switching contexts. Explicitly state when you are uncertain, when something may be hallucinated, or when sources conflict. List concrete examples or exercises that illustrate differences. If a topic is unclear or ambiguous, you must request clarification rather than fabricate.

12/8/2025, 3:25:12 AM

I understand. I'm ready to provide precise, comparative analysis of overlapping topics across university courses. I will:

1. **Compare specific concepts** - not just list what each course covers, but analyze how they treat the same material differently
2. **Identify pedagogical differences** - in emphasis, definitions, assumptions, notation, and problem-solving approaches
3. **Highlight conflicts** - where courses may teach contradictory or incompatible perspectives
4. **Provide context-switching guidance** - help students understand how to adapt their thinking between courses
5. **Be transparent about uncertainty** - clearly flag when I'm unsure, when information might be incomplete, or when I'm making inferences
6. **Use concrete examples** - provide specific problems, exercises, or scenarios that illustrate the differences
7. **Request clarification** - ask questions when topics are ambiguous rather than assume

To get started, please provide:

- The courses you want compared (names, numbers, or descriptions)
- Specific topics or concepts you want analyzed
- Any relevant materials (syllabi, lecture notes, problem sets, textbooks) if available

I'll deliver analysis that helps you navigate the conceptual landscape across your courses with precision and honesty about the limits of my knowledge.

12/8/2025, 3:25:20 AM



File: EECS_183_283a_Posttraining.pdf

File: Lecture 24.pdf

File: Lecture 25.pdf

File: Lecture 26.pdf

I am taking the following courses this semester: Course A: EECS 183 - Natural Language Processing Course B: EECS 182 - Deep Neural Networks I want you to compare the following topic(s) across these classes: Topic(s): llm post-training Please perform the comparison using the following structure:

1. Definitions & Formalism How each class defines the concept Notation differences Underlying assumptions
2. Conceptual Goals of Each Course Why each class teaches this concept Depth vs breadth Whether each course treats the concept theoretically, heuristically, or operationally
3. Methodological Differences Algorithms emphasized in each Differences in derivations or interpretations Problem-solving styles expected in homework/exams
4. Conflicts or Divergences Common student misconceptions when transferring knowledge between courses Where the classes disagree in framing or intuition
5. Complementary Strengths What perspective each course adds How understanding from one course deepens knowledge in another
6. Study Strategy Recommendations How to integrate the perspectives How to avoid context-specific misunderstandings How to create a unified mental model
7. Hallucination Check For each major claim: Mark [Certain], [Plausible but check course materials], or [Uncertain]. If you are unsure about anything or lack information, state it explicitly instead of guessing.



Comparative Analysis: LLM Post-Training across EECS 183 (NLP) and EECS 182 (Deep Neural Networks)

Based on the provided course materials, I'll compare how these two courses approach LLM post-training.

1. Definitions & Formalism

EECS 183 (NLP) - From Post-training PDF

Definition: Post-training is presented as a two-stage process to address:

- **Interface problem:** Making models respond to natural language instructions rather than text completion
- **Alignment problem:** Controlling what models should/shouldn't generate

Key Components:

1. **Instruction Tuning:** Fine-tune to produce responses conditioned on natural language instructions
 - Uses instruction data: (instruction, demonstration) pairs
 - Loss function: Standard cross-entropy on demonstrations only (masked loss on prompts)
2. **RLHF (Reinforcement Learning from Human Feedback):**
 - Train reward model $r: X \times Y \rightarrow \mathbb{R}$ from preference data

- Reward model loss: $L(\theta) = -1/(N \text{ choose } 2) \times E[\log \sigma(r(x, y_w; \theta) - r(x, y_l; \theta))]$
- RL objective with KL regularization with instruction-tuned model

Notation:

- $D_{\text{instruct}} = \{(x^{(i)}, y^{(i)})\}$
- Preference data: $D_{\text{pref}} = \{(x, \tilde{y}_w, \tilde{y}_l)\}$
- Policy π , reward r , reference policy π_{ref}

EECS 182 (Deep Neural Networks) - From Lecture Notes

Definition: Appears more focused on the **test-time compute** and **inference-time optimization** aspects

Key Components (from Lecture 25-26):

1. Reinforcement Learning with Verifiable Rewards (RLVR):

- Focuses on problems where outcomes can be verified
- Objective: $J_{\text{ScaleRL}}(\theta)$ with generation scheduling and pass_rate constraints
- Emphasis on scaling laws and compute efficiency

2. Test-time approaches:

- Best-of-N sampling
- Beam search with learned reward models
- MCMC-based sampling for better exploration

Notation:

- More emphasis on computational scaling (GPU hours, pass rates)
- Distribution mismatch concepts
- Normalized rewards: $A^{\text{norm}} = \hat{A}_i / A_{\text{std}}$

Assumptions:

- **EECS 183:** Assumes access to human preferences; treats post-training as adapting pretrained capabilities
- **EECS 182:** Assumes verifiable rewards exist; focuses on compute-optimal strategies

[Certain - based directly on provided materials]

2. Conceptual Goals of Each Course

EECS 183 (NLP)

Why taught:

- To understand the full modern LLM pipeline: pretraining → post-training
- To address practical deployment challenges (interface, alignment)
- Part of understanding how models like ChatGPT are created from base models

Depth: Moderate depth on:

- Human feedback collection methodology
- Reward model training mechanics
- Basic RL formulation (policy gradient/REINFORCE)

I think these high level goals are important to know but maybe the prompt overemphasizes showing these instead of technical details. However this could be due to my sparse 183 context.

Treatment:

- **Operational/Applied:** Focus on "what practitioners do"
- Covers data collection, annotation practices
- Discusses practical issues (reward hacking, hallucination, off-policy problems)

EECS 182 (Deep Neural Networks)

Why taught:

- To understand scaling properties and compute allocation
- To explore test-time optimization as model improvement
- Part of broader coverage of training strategies

Depth: Deep focus on:

- Scaling laws for RL compute
- Mathematical foundations of sampling strategies
- Computational efficiency trade-offs

Treatment:

- **Theoretical + Empirical:** Emphasizes scaling curves, mathematical derivations
- Focus on "how much compute" and "what sampling strategy"
- Less emphasis on data collection, more on algorithmic efficiency

[Certain]

3. Methodological Differences

EECS 183 (NLP)

Algorithms Emphasized:

1. **Instruction Tuning:** Standard supervised fine-tuning with masked loss
2. **Basic RLHF:**
 - Reward model training with pairwise preferences
 - Policy gradient (REINFORCE) with KL penalty
3. **Data Collection:** How to gather instruction data and preferences

Problem-Solving Style:



- "Given this scenario, design an instruction-tuning dataset"
- "Identify potential reward hacking issues"
- Understanding trade-offs in human annotation

EECS 182 (Deep Neural Networks)

Algorithms Emphasized:

1. RLVR (Reinforcement Learning with Verifiable Rewards):

- Generation scheduling across G generators
- Pass rate requirements
- Distribution normalization

2. Test-Time Compute Strategies:

- Best-of-N with various selection criteria
- Beam search variants
- MCMC/proposal optimization
- **Power sampling for autoregressive models:**
 - Low-temperature sampling from $p(x)^\alpha$
 - Addresses non-differentiable losses

3. Scaling Analysis:

- Fitting curves: $R_C = R_0 + A/(1+(C_{\text{mid}}/C)^B)$
- Computing optimal compute allocation

Problem-Solving Style (from lecture notes):

- "Given compute budget C, optimize sampling strategy"
- "Derive acceptance ratios for MCMC proposals"
- "Analyze scaling exponent B to predict performance"

[Certain for EECS 183, Plausible but check for EECS 182 - based on lecture notes but not exam materials]

4. Conflicts or Divergences

Framing Differences

What is "post-training"?

- **EECS 183:** Post-training = instruction tuning + RLHF as a **necessary pipeline stage**
- **EECS 182:** Post-training appears to emphasize **test-time compute** and **verifiable rewards** - less focus on human preferences per se

Common Student Misconceptions

1. "RLHF always requires human feedback"

I think this is an important perspectival difference acknowledged

- EECS 183 framing: Yes, human preferences are central
- EECS 182 framing: RLVR uses *verifiable* rewards (e.g., code correctness) - humans may not be in the loop
- **Conflict:** Students may think all RL post-training needs human annotation

2. "The goal is to maximize reward"

- EECS 183: Emphasizes KL regularization to prevent drift from base model
- EECS 182: Focuses on `pass_rate < 0.9` and distribution normalization constraints
- **Conflict:** Naive reward maximization vs. constrained optimization

3. "More compute always helps"

- EECS 183: Doesn't deeply address compute scaling

- EECS 182: Explicitly shows scaling curves plateau; emphasizes "bag of tricks parameter"
- **Conflict:** Students may not realize diminishing returns without 182's perspective

4. "Sampling strategy doesn't matter"

- EECS 183: Basic mention of on-policy training needs
- EECS 182: Detailed treatment of low-temp sampling, MCMC, Best-of-N
- **Conflict:** 183 students may underestimate sampling sophistication

Notation Conflicts

- $r(x,y)$ in EECS 183: Reward model output (scalar)
- **EECS 182** uses $p_{\{i,t\}}$ for normalized rewards with generator-specific indexing
- Students must track which "reward" concept is being discussed

[Certain on first two misconceptions, Plausible on others]

5. Complementary Strengths

EECS 183 (NLP) Strengths

1. **Human-in-the-loop perspective:** Understanding preference collection, annotation quality
2. **Alignment framing:** Why we care about controlling outputs (safety, helpfulness)
3. **Data quality:** How instruction data diversity matters
4. **Practical deployment:** Covers issues like reward hacking, hallucination

EECS 182 (DNN) Strengths

1. **Computational perspective:** When to use more samples vs. better sampling
2. **Scaling laws:** Predictable performance improvements with compute
3. **Test-time optimization:** Treating inference as an optimization problem
4. **Mathematical rigor:** Derives sampling strategies from first principles

How They Complement

Example: Understanding RLHF performance

- **183 tells you:** What preference data to collect, how to train reward model
- **182 tells you:** How much compute to allocate, what sampling strategy during RL training

Example: Improving model outputs

- **183 tells you:** Use RLHF to align with human preferences
- **182 tells you:** Or use Best-of-N at test time with a learned verifier - same effect, different compute allocation

Unified view: Post-training is about **shaping the output distribution** - you can do this via:

- Training-time adaptation (183's focus)
- Test-time compute (182's focus)

[Certain]

6. Study Strategy Recommendations

Integration Strategy

1. **Map concepts explicitly:**

EECS 183 RLHF pipeline:

Pretrain → Instruction-tune → Reward model → RL optimization

EECS 182 RLVR pipeline:

Pretrain → SFT → RLVR with verifiable rewards + test-time compute

Common structure: Base model → Supervised adaptation → RL with rewards

Interesting to see how it unifies the two perspectives

2. Distinguish contexts:

- **Human preferences** (183) vs. **Verifiable outcomes** (182)
- **Training-time RL** (183) vs. **Test-time optimization** (182)
- Both are valid; choice depends on problem

3. Create unified mental model:

Post-training goal: Shape $p(y|x)$ to match desired behavior

Methods:

- Supervised: Instruction tuning (direct demonstration)
- RL-based: RLHF (human preferences) or RLVR (verifiable rewards)
- Test-time: Best-of-N, beam search (amortize compute at inference)

Trade-off: Training cost vs. inference cost

Avoid Context-Specific Misunderstandings

1. When seeing "reward model":

- Ask: "Is this learned from preferences (183) or computed from verification (182)?"

2. When seeing "RL optimization":

- Ask: "Is this training the policy (183) or sampling at test time (182)?"

3. When seeing scaling discussions:

- 183: More about data diversity and quality
- 182: More about compute hours and algorithmic efficiency

4. For exams:

- **183**: Focus on pipeline stages, data collection, alignment problems
- **182**: Focus on scaling curves, sampling algorithms, compute allocation

Study Tactics

1. **Make comparison tables** like this analysis
2. **Work problems from both courses** on same day to see differences
3. **Explain to yourself**: "How would 183 solve this? How would 182 solve this?"
4. **Track notation carefully** - write out what each symbol means in each course

[Certain on strategy, Plausible on specific tactics]

7. Hallucination Check & Explicit Uncertainties

What I'm Certain About

- ✓ EECS 183 covers instruction tuning and RLHF as two-stage post-training [from PDF pages 6, 18-19, 33-38]
- ✓ EECS 182 emphasizes RLVR and scaling laws [from lecture 25-26 pages 2-3, 5-

6]

- ✓ The courses have different emphases: human preferences vs. verifiable rewards
- ✓ EECS 182 includes detailed test-time compute strategies
- ✓ Notation and problem-solving styles differ between courses

What I'm Uncertain About

- ⚠ **EECS 182 exam expectations:** I don't have exam materials for 182, only lecture notes - actual exam emphasis may differ
- ⚠ **Depth of coverage:** Lecture notes may not reflect full treatment in readings or projects
- ⚠ **DPO (Direct Preference Optimization):** Mentioned in 183 materials but not fully detailed - unclear if 182 covers this
- ⚠ **Specific homework problems:** Don't have access to problem sets, so problem-solving style is inferred

What I Cannot Determine from Materials

- ✗ Whether 182 covers human preference-based RLHF at all (materials focus on verifiable rewards)
- ✗ Full scope of 182's RL coverage - may include more standard RL than visible in these notes
- ✗ Whether students take these courses concurrently or sequentially - affects pedagogical assumptions
- ✗ Specific textbook chapters or readings that might bridge the perspectives

Recommendations for Verification

1. **Check course syllabi** to see complete topic coverage
2. **Review homework assignments** from both courses for actual problem styles
3. **Attend office hours** to ask about connections between courses
4. **Look at past exams** (if available) to understand evaluation emphasis

Summary Table

Aspect	EECS 183 (NLP)	EECS 182 (DNN)
Primary focus	Human alignment pipeline	Compute-efficient optimization
Data source	Human preferences	Verifiable rewards
When optimizing	Training time (RL)	Training + test time
Key metric	Alignment quality	Compute efficiency / scaling
Reward source	Learned from comparisons	Computed from verification
Typical problem	"Design annotation pipeline"	"Allocate compute budget"
Mathematical depth	Moderate (basic RL)	Deep (scaling laws, MCMC)
Practical emphasis	Deployment & safety	Algorithmic efficiency

[Certain on this table given available materials]

Final Note: This analysis is based solely on the provided lecture materials. For a complete picture, I recommend:

- 1. Reviewing full course syllabi
- 2. Checking if there are cross-listed prerequisites or recommended background
- 3. Asking instructors directly about how the courses relate
- 4. Looking at final projects from both courses to see integration points

12/8/2025, 3:30:19 AM

Overall I think this conversation history is fine insofar as I didn't notice anything wrong. This one wasn't as helpful as my other one that gave me a concrete plan for simultaneous study of 180 and 182 though.